College Helpdesk Chatbot

Project Report

Name :	Anamika Singh
Reg. NO :	25BCE10247
Corse Code :	CSA2001
Faculty :	Prakash N B
Date :	31/03/2026

1. Introduction

With the growing number of queries students raise every day regarding college-related information - fee structures, exam schedules, placements, hostel facilities, and more - it becomes essential to provide quick and accurate responses without placing excessive burden on administrative staff. Manual handling of such queries is time-consuming, inconsistent, and often unavailable outside working hours.

The College Helpdesk Chatbot is a Python-based conversational assistant designed to handle frequently asked questions from students in an automated, intelligent manner. By leveraging Natural Language Processing (NLP) techniques - specifically TF-IDF vectorization and Cosine Similarity - the chatbot is able to match user queries to a pre-defined knowledge base and return the most relevant answer.

This project aims to:

- Reduce dependency on college staff for routine information queries
- Provide 24/7 availability for student assistance
- Offer a seamless escalation path to a human executive when needed
- Serve as a lightweight, cost-effective solution that requires no complex AI infrastructure

The system is modular, text-driven, and easily extensible - making it an ideal starting point for building smarter institutional helpdesks.

2. Problem Statement

Every academic institution receives a high volume of repetitive queries from students and prospective applicants. These queries range from fee-related questions to exam schedules, placements, hostel availability, scholarship information, and more. The challenges include:

- Staff are repeatedly answering the same questions, leading to inefficiency
- Students may not get timely responses outside office hours
- No centralized, self-service channel exists for basic information
- Lack of a mechanism to escalate unresolved queries to the right person

This project addresses the above problems by building an automated, intelligent, text-matching chatbot that can serve students instantly, with a fallback mechanism to connect them to a college executive when the chatbot cannot answer.

3. Approach to Solving the Problem

3.1 Data Preparation (data.txt)

A structured flat-file knowledge base (data.txt) was created containing question-answer pairs. Each line follows a simple key: value format, covering topics such as fees, placements, library timings, hostel, canteen, WiFi, scholarships, events, and more. This makes the knowledge base easy to update without touching the code.

3.2 File Handling (file_handler.py)

A dedicated module loads the knowledge base into a Python dictionary at runtime. It also provides a function to persist user contact details to a separate file (user_details.txt) when a user opts to connect with a college executive.

3.3 NLP-Powered Matching (chatbot.py)

The core intelligence of the chatbot lies in this module. It uses two NLP techniques:

- TF-IDF Vectorization (TfidfVectorizer from scikit-learn): Converts all questions in the knowledge base, along with the user's query, into numerical vectors based on word frequency and importance.
- Cosine Similarity: Measures the angular similarity between the user's query vector and each question vector. A score above 0.3 is treated as a confident match.

This approach is language-tolerant - it handles partial matches, synonyms in phrasing, and minor spelling variations, making it much more robust than simple keyword matching.

3.4 Conversational Flow (main.py)

The main module orchestrates the chatbot flow: greeting the user, accepting queries in a loop, displaying responses, and offering the option to ask more questions. If the chatbot fails to find a match, it proactively offers to connect the user with a college executive and collects their contact information.

4. Key Decisions Made

- TF-IDF over Keyword Matching: Pure keyword matching would fail for paraphrased or partial queries.TF-IDF provides weighted importance to words, making it more semantically aware.
- Flat-file Knowledge Base: Using a plain .txt file for the Q&A data makes the system easy to maintain and update, without needing a database or admin panel.
- Threshold of 0.3: A cosine similarity score of 0.3 was chosen as the minimum confidence threshold to balance between returning irrelevant results and rejecting valid queries.

- Modular Code Design: The project was split across three modules (chatbot.py, file_handler.py, main.py) to separate concerns - NLP logic, I/O handling, and conversational flow - improving maintainability.
- Executive Escalation: A built-in path allows unresolved queries to escalate gracefully to a human, ensuring no student is left without help.

5. Challenges Faced

- Ambiguous Queries: Students often phrase questions vaguely (e.g., 'fees?' or 'when exams?'). The TF-IDF model handles such brevity reasonably, but very short queries can score below threshold even when relevant.
- Colon Parsing Issue: The file_handler.py splits each line on the first colon (:) only. Answers containing colons (e.g., times like '9 AM: to 5 PM') were initially causing parsing errors - resolved by ensuring answers avoid embedded colons, or by adjusting the split strategy.
- No Memory/Context: The chatbot treats each query independently with no conversational memory. Follow-up questions like 'What about hostel fees?' cannot leverage prior context.
- Limited Knowledge Base: The system can only answer what is explicitly present in data.txt. Any question outside the defined scope returns a 'Sorry, I couldn't understand' response.
- Case Sensitivity: All inputs are lowercased, but typos or misspellings still affect match quality since TF-IDF is token-based.

6. What Was Learned

This project provided hands-on experience with several important concepts and skills:

- NLP Fundamentals: Understanding how TF-IDF vectorization works, and how cosine similarity enables semantic text comparison without requiring deep learning models.
- Modular Python Design: Structuring a Python project across multiple files with clear separation of responsibilities - improving readability, testability, and maintainability.
- Practical Trade-offs: Learned to balance simplicity vs. sophistication. A rule-based, NLP-enhanced system can be highly effective for constrained domains without the overhead of training a neural network.
- User Experience Considerations: Designing a chatbot that gracefully handles failure states (unknown queries) and provides an alternative resolution path (executive escalation) is as important as the core matching logic.
- Data-Driven Systems: Even a simple flat file can power a meaningful application when the data is well-structured and the retrieval mechanism is intelligent.

7. System Architecture Overview

The system follows a simple three-layer architecture:

- Data Layer: data.txt (knowledge base) and user_details.txt (escalation records)
- Logic Layer: chatbot.py (NLP matching), file_handler.py (file I/O utilities)
- Presentation Layer: main.py (CLI-based conversational interface)

The flow on each user query is: Input - Lowercase - TF-IDF Vectorize - Cosine Similarity - Best Match - Response. If the similarity score falls below 0.3, the chatbot flags the query as unresolved and triggers the escalation flow.

8. Future Scope

- Web/WhatsApp Interface: Port the chatbot to a web-based or messaging platform (e.g., Flask + WhatsApp API) for broader accessibility.
- Intent Classification with ML: Train a lightweight classifier (e.g., Naive Bayes or BERT) to handle more complex, multi-intent queries.
- Context Awareness: Maintain conversation history to support follow-up questions.
- Admin Panel: Build a simple dashboard to update the knowledge base without editing raw text files.
- Multi-language Support: Add support for regional languages to serve a wider student base.

9. How to run the project

9.1. Method

- Python 3.x installed

9.2. Steps

- Clone the repository from GitHub
- Navigate to the project directory
- Run the main script: python main.py

10. Conclusion

The College Helpdesk Chatbot successfully demonstrates how NLP techniques - even without deep learning - can be used to build a practical, useful conversational system. By combining TF-IDF vectorization with cosine similarity, the chatbot is able to intelligently match student queries to relevant answers from a structured knowledge base, providing instant responses on topics ranging from fees and placements to events and facilities.

The modular design ensures that the project is easy to extend and maintain. The built-in escalation mechanism ensures that students are never left without assistance, even when the chatbot reaches the limits of its knowledge.

This project reinforced the value of thoughtful system design, NLP fundamentals, and user-centric thinking. It serves as a solid foundation for building more sophisticated AI-powered helpdesk systems in the future - one that can grow from a command-line prototype to a full-scale, multi-channel institutional assistant.